

SPARK-V: Small Power-Efficient Architecture for RISC-V

1st Justin Lebeau

Electrical and Computer Engineering

Rice University

Houston, Texas, United States

jhl6@rice.edu

Abstract—Wearable biomedical devices require processors that balance performance with strict energy constraints, as battery life directly impacts usability and reliability in real world health monitoring. Current solutions often rely on proprietary low power processors, which can be expensive and lack flexibility for customization. Our project proposes an open-source alternative: developing and validating a low power RISC-V processor tailored for biomedical applications. We begin by validating an existing RISC-V core on an FPGA platform to demonstrate full functionality and suitability for wearable workloads. Building on this foundation, we target a 30% reduction in system power through architectural and implementation-level optimizations. Ultimately, this project aims to produce a validated, energy efficient processor ready for tapeout, enabling more accessible and customizable solutions for biomedical wearables.

Index Terms—RISC-V, Low-Power Design

I. INTRODUCTION

The design goal of this project is to create a processor for use in wearable biomedical devices. Since these devices are battery-powered, they require an architecture that is as low-energy as possible to maximize device usability and reliability.

While low-power processors currently exist, many of these are proprietary, which often makes them restrictive, expensive, or challenging to customize. To address this, the SPARK-V project proposes an open-source alternative using the RISC-V instruction set. The project begins by utilizing WALLY, an existing open-source RISC-V core from Harvey Mudd College, as a baseline [1].

The primary goal is to achieve significant power efficiency, specifically targeting a 30% reduction in system power. This will be accomplished through systematic architectural and implementation-level power optimizations, culminating in a validated, energy-efficient processor.

II. METHODS

This project will be broken across two semesters, and thus I designed a set of goals for each semester. This semester, I first needed to set up a Virtual Machine with all the needed tools for design and verification. Then, I wanted to get WALLY to run on an FPGA. The Github page has instructions for booting Linux on an FPGA, and this was my first goal in validating the functionality of the current design. After this was completed, my next the goals were to set up a benchmarking specification to evaluate processor performance, then use this benchmark

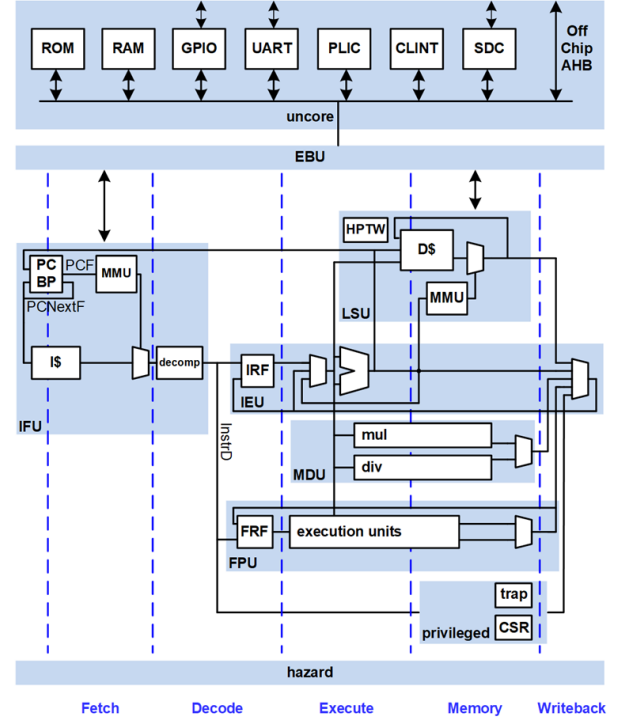


Fig. 1. Block Diagram of the WALLY system [1]

to compare different architectures and choose an architecture to use for further study. This will allow me to work on more physical design aspects next semester for the chosen architectural configuration.

III. RESULTS

A. Development Environment Setup

The initial work in developing WALLY requires a reliable development environment. For this purpose, I set up a Virtual Machine (VM) on the ORION system. This setup is complete, and I am able to see the source files and run verification tests. I created an instructional document detailing the steps required to set up the WALLY project for development for future users. The process was tested by the VLIW Senior Capstone Team, who were able to provide feedback to improve my instructions. The WALLY core itself is available through an

open-source GitHub repository [1]. In addition to the GitHub repository, the VM contains open-source and proprietary chip design software. This includes open-source RISC-V compilers, and Verilator, an open-source SystemVerilog simulator. It also includes Questa, Design Compiler, and Vivado, commercial chip design tools. I have verified that all of these tools function in the VM environment, and have created specific setup instructions for Design Compiler, as this requires more work beyond the initial setup.

B. WALLY on an FPGA

In order to implement WALLY on an FPGA, I first had to decide on a specific board to use. The WALLY GitHub provides three options for FPGA implementations, but the most affordable option is the Arty A7 100T FPGA Board from Digilent. This board contains an AMD Artix-7 FPGA, as well as the needed connections for testing, including JTAG and UART support, as well as PMOD connectors that can be used to access an SD card [2].

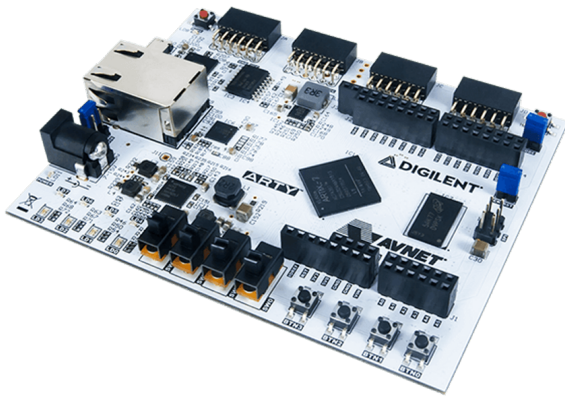


Fig. 2. The Arty A7 100-T FPGA Board [2]

With this board chosen, I was able to use Vivado to generate a bitfile to program the FPGA with. The WALLY GitHub provided a script to flash an SD card with a Linux Kernel image, but since I was using a VM I could not directly flash it. So, I modified the script to write the output to a file, which I was then able to download and flash from my laptop. I also copied the Vivado project to my computer to program the FPGA.

I was able boot Linux on the FPGA, and running basic commands to demonstrate that it was responsive and functional. Screenshots showing the FPGA board booting Linux and responding to commands can be seen below.

This was a valuable exercise to show the functionality of the full WALLY design. However, our design mandates that

Fig. 3. Linux Booting and Running on WALLY

we use a much less complex architecture, that will run without an operating system. Therefore, a testing plan for new designs requires a different platform testing. To test several designs for both logical correctness and power usage, I have started in simulation. Later, I can evaluate the best approach to validate this in emulation on the FPGA.

C. Design Specifications

Our goal is to create a fully functional RISC-V processor with ultra low-power usage. In order to minimize power usage, we will need to cut down as many extensions as possible while still keeping the core functionality of the chip. The current design plan specifies the following features:

- A **single-core, single-threaded processor**.
- An **integer-only** instruction set.
- Inclusion of the **base instruction set**.
- Inclusion of the **compressed (C)** instruction set to reduce code memory usage.
- Inclusion of the **minimal multiplication (Zmmul)** extension. This implements multiplication but not division, allowing for increased performance while avoiding the full area requirement of division circuitry.

The design specification for the SPARK-V core maintains a high degree of flexibility, prioritizing power efficiency while allowing for targeted performance enhancements. Additional components, such as a dedicated division unit or an instruction/data cache, may be incorporated if testing determines their

benefits outweigh the marginal power cost. The decision to include a division unit, for instance, will depend on whether its performance gain in critical biomedical signal processing routines justifies the increased complexity and area overhead, especially since multiplication is already included. Similarly, the incorporation of a cache will be evaluated by comparing its power consumption against the energy savings achieved by reducing external memory accesses. This iterative process, which involves comparing the power usage of several different designs, ensures that the final architecture remains energy-conscious. Therefore, the core will only adopt new extensions or subsystems that offer a substantial, application-specific return on investment in terms of power-to-performance ratio. This also means that future testing will require some examples of programs and routines that are common in biomedical sensors, so that functional unit usage can be measured.

D. Benchmarking Using Embench

As previously mentioned, testing power and performance requires a set of benchmarks that mimic real embedded workloads. For this purpose, I decided to use Embench, a benchmark set that is specifically targeted to embedded devices without an operating system [3]. Embench contains 19 programs, designed to stress the compute, branching, and memory functionality of the processor differently.

Each program is compiled with two different optimizations: one for maximum speed, and one for minimum code size. The programs are then run, and code size and program speed are measured for both variations. These size and speed values are then compared to the performance on the ARMv7-M processor, which is used as the Embench baseline [3]. Values lower than 1.00 indicate smaller code / slower execution, while values greater than 1.00 indicate larger code / faster execution. The overall score is given by the geometric mean of each program score.

In order to simplify my analysis, I decided to only evaluate the size and speed of the size optimized code. I expect the final design to have a minimal memory size, and thus want my testing to focus on smaller program sizes over performance enhancements. Figure 4 shows the results of the size optimized program for four configurations of the processor. Embench uses control and status registers to measure processor cycles and instructions retired, and thus all tests using Embench require the use of the zicsr extension.

Configuration	Mean Size	Size Std. Deviation	Mean Speed	Speed Std. Deviation
RV32IMC_Zicsr	0.95	1.29	1.07	1.52
RV32I_Zmmul_C_Zicsr	1.04	1.26	0.79	2.07
RV32IC_Zicsr	1.06	1.25	0.47	2.73
RV32I_Zicsr	1.52	1.28	0.47	2.73

Fig. 4. Embench Size-Opt Results for Various Architectures

Looking at the size results, we can see that the biggest variation is between the architectures that use the compressed instruction (C) extension, and the one that does not. Not

having compressed instructions increases the code size by almost 50%, meaning that the majority of instructions used by Embench are able to be compressed. Adding multiplication (Zmmul) does not seem to have a significant impact on code size, but division and multiplication (M) does provide an approximate 10% reduction in code size over Zmmul and no multiplication.

For the speed results, we can see that the IMC extensions allow for the best speed, which makes sense as several workloads in Embench contain division. In figure 5, we can see the individual speed results for each program in the RV32IMC_Zicsr and RV32I_Zmmul_C_Zicsr configurations.

```
{
  "speed results" :
  {
    "detailed speed results" :
    {
      "aha-mont64" : 0.82,
      "crc32" : 1.00,
      "cubic" : 0.39,
      "edn" : 0.94,
      "huffbench" : 1.38,
      "matmult-int" : 1.11,
      "md5sum" : 1.99,
      "minver" : 0.64,
      "nbody" : 0.68,
      "nettle-aes" : 0.81,
      "nettle-sha256" : 0.96,
      "nsichneu" : 1.14,
      "picojpeg" : 0.79,
      "primecount" : 1.30,
      "qduino" : 1.23,
      "sglib-combined" : 1.17,
      "slre" : 1.26,
      "st" : 0.86,
      "statemate" : 2.16,
      "tarfind" : 2.42,
      "ud" : 0.88,
      "wikisort" : 1.76
    },
    "speed geometric mean" : 1.07,
    "speed geometric standard deviation" : 1.52
  }
}

{
  "speed results" :
  {
    "detailed speed results" :
    {
      "aha-mont64" : 0.82,
      "crc32" : 1.00,
      "cubic" : 0.13,
      "edn" : 0.94,
      "huffbench" : 1.38,
      "matmult-int" : 1.11,
      "md5sum" : 1.98,
      "minver" : 0.27,
      "nbody" : 0.18,
      "nettle-aes" : 0.77,
      "nettle-sha256" : 0.96,
      "nsichneu" : 1.14,
      "picojpeg" : 0.79,
      "primecount" : 1.30,
      "qduino" : 1.23,
      "sglib-combined" : 1.11,
      "slre" : 1.25,
      "st" : 0.22,
      "statemate" : 2.16,
      "tarfind" : 0.76,
      "ud" : 0.86,
      "wikisort" : 0.80
    },
    "speed geometric mean" : 0.79,
    "speed geometric standard deviation" : 2.07
  }
}
```

Fig. 5. Individual Embench Program Speed Results With Division (top), and without Division (bottom).

Reducing the set from M to Zmmul reduces performance by about 25%, showing the value of having a divider unit. However, we can also see the standard deviation increases greatly, which makes sense. Only 6 of the 19 embench programs use division if available, meaning that only these programs experience the performance improvement of the divider unit. This causes the resultant performance of the programs to have greater variation.

The 6 programs that use division are cubic, minver, nbdiv, st, tarfind, and wikisort. Comparing the speed results of these programs with and without the use of a divider unit, we can see execution speed is reduced by a factor of 2.2-3.9, depending on the specific program and how frequently the division instruction is used.

Removing multiplication further reduces performance, dropping from 0.79 to 0.47 (value compared to ARMv7-M processor). The use of compressed instructions does not change the speed of execution, which is expected.

Overall, this shows the importance of at least the inclusion of compressed instructions for code size and multiplication for code speed. While division is useful in further improving performance for some workloads, the inclusion of division will depend on the expected workloads our processor will use, as programs without division obviously do not benefit from it, and multiplication-based workarounds can be implemented in software if division is necessary. Currently, we expect the programs that will run on this processor will avoid division in almost all cases. Assuming this continues to be true, the divider will likely not be included in the final design. However, this may change in the future.

E. Peripherals

In addition to these decisions that affect the core of WALLY, I also need to decide how to manage the peripheral connections WALLY contains. As per the GitHub project, these components are referred to as the "uncore" module. These include ROM, RAM, timers, GPIO, and communication over UART. While the design of the core has been specified above, additional work will be needed to decide which uncore components will be used, and what settings will be applied to them.

Currently, it makes sense to include all of these peripherals to some extent. Program ROM and RAM are fundamental for any standalone system, and the inclusion of PLIC and CLINT enables timer-driven and interrupt-driven execution, both of which are standard requirements for embedded workloads. UART, SDC, and GPIO interfaces also provide practical pathways for off-chip communication, data transfer, and basic system control, making them useful for both development and deployment. In practice, the exact set of peripherals will depend on system constraints. For example, GPIO is flexible and broadly applicable, but the number of exposed pins may need to be reduced if the final processor package has limited I/O availability. Careful selection and scaling of these peripherals will help ensure functionality without exceeding physical or architectural constraints.

IV. DISCUSSION

At this point in the year, I have been able to set up the needed development environment on ORION and validate the base WALLY design by using it to boot Linux on an FPGA. I also defined an initial architectural specification with the goal of maximizing power-efficiency for wearable biomedical applications. This plan focuses on a minimalist design through the reduction of extensions and add-ons. The current specification is a single-core, single-threaded, integer-only processor utilizing the base instruction set, the compressed (C) extension to reduce code size, and the multiplication (Zmmul) extension (excluding division) for performance. The next focus is to test and finalize the best architectural configuration using compiled designs rather than just RTL simulation. This requires the choice of a Standard Cell Library and the use of Design Compiler, and will give me further ability to compare the power usage of key design elements, such as caches and branch prediction.

Looking ahead to next semester, the project will move into the physical design phase. The primary goal is to start from the optimized architectural configuration and apply implementation-level decisions to achieve the overall objective of a 30% reduction in system power. Ultimately, the aim is to produce a validated, energy-efficient processor ready for tapeout. I am also looking into additional power-reduction possibilities, including a paper that proposes an entirely battery-less RISC-V processor, which would be an extremely interesting implementation if possible with this design [4]. This could allow for a truly mobile processor for medical applications, but may introduce new challenges.

ACKNOWLEDGMENT

I would like to thank Dr. Ray Simar for his support on this project, as well as Dr. James Stine and Jacob Pease from Oklahoma State University for their continued guidance. I would also like to thank David Harris and Sarah Harris for granting pre-publication access to their textbook covering the WALLY design which has helped in understanding the processor. Additionally, thanks to the Rice Electrical and Computer Engineering Department for funding this project.

CREDIT AUTHOR STATEMENT

Justin Lebeau: Conceptualization, Investigation, Methodology, Software, Validation, Writing

REFERENCES

- [1] <https://github.com/openhwgroup/cvw>
- [2] <https://digilent.com/shop/arty-a7-100t-artix-7-fpga-development-board/>
- [3] <https://github.com/embench/embench-iot>
- [4] D. S. Truesdell, J. Breiholz, S. Kamineni, N. Liu, A. Magyar and B. H. Calhoun, "A 6–140-nW 11 Hz–8.2-kHz DVFS RISC-V Microprocessor Using Scalable Dynamic Leakage-Suppression Logic," in *IEEE Solid-State Circuits Letters*, vol. 2, no. 8, pp. 57-60, Aug. 2019, doi: 10.1109/LSSC.2019.2938897.